LOW LEVEL DESIGN DOCUMENT

CI/CD Pipeline for Vue.js Application

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

S. No.	Student Name	Enrollment Number
1.	Aayush Pal	EN22CS301022
2.	Aayushee Dhote	EN22CS301024
3.	Aditi Shrestha	EN22CS301044
4.	Aditya Chouhan	EN22CS301049
5.	Aditya Singh Chouhan	EN22CS301066

Group Name: Group 09D1

Project Number: DO-09

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 2025 - 26

Table of Contents

1. Introduction
    - 1.1 Scope
    - 1.2 Intended Audience
    - 1.3 Objectives
    - 1.4 System Overview
2. System Design (LLD)
    - 2.1 Application Design
    - 2.2 Process Flow
    - 2.3 Information Flow
    - 2.4 Component Design
    - 2.5 Key Design Considerations
    - 2.6 API Catalogue
3. Tools Used
4. References

1. Introduction

In modern software development, delivering applications quickly and reliably has become a critical requirement. Traditional manual deployment methods are time-consuming, error-prone, and inconsistent, often delaying release cycles and reducing software quality.

To address these issues, **DevOps practices** such as **Continuous Integration (CI)** and **Continuous Deployment (CD)** are adopted. CI/CD automates integration, testing, and deployment, reducing errors and increasing consistency.

This project focuses on designing and implementing a **CI/CD pipeline for a Vue.js application** using modern tools like **GitHub, GitHub Actions, Docker, and Kubernetes** to automate build, test, containerization, and deployment.

1.1 Scope

The scope of this project includes the development of a complete CI/CD pipeline that integrates version control, automated testing, containerization, and deployment. It covers the use of Git-based repositories for source code management, CI/CD tools for automation, Docker for container creation, and Kubernetes for deployment and scaling.

The project focuses on automating the lifecycle from code commit to deployment. However, it does not include advanced cloud configurations, multi-region deployment, or complex monitoring and security compliance systems.

1.2 Intended Audience

- Developers working on the Vue.js application
- DevOps Engineers managing CI/CD pipelines
- Evaluators, mentors, and instructors reviewing this project

1.3 Objectives

The main objectives of this project are to:

- Automate the integration and deployment process
- Ensure code quality through automated testing
- Reduce deployment time and manual effort
- Enable containerized application deployment
- Provide scalability and reliability using Kubernetes
- Improve consistency across development and production environments

1.4 System Overview

The end-to-end CI/CD process is as follows:

Code Commit → Build → Test → Dockerize → Deploy

Each stage is triggered automatically when code changes are pushed to the repository.

2. System Design (LLD)

2.1 Application Design

Frontend Layer

- Developed using Vue.js
- Static build generated via npm run build

CI/CD Layer

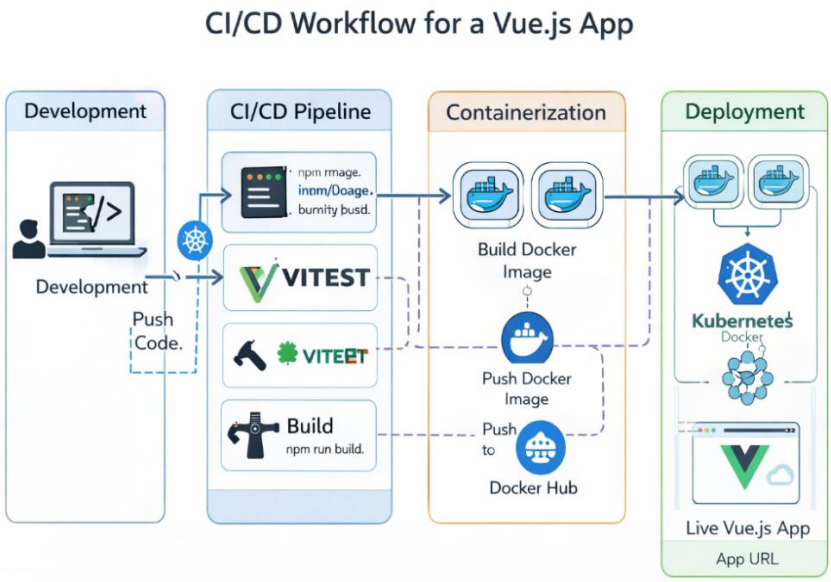
- Managed using GitHub Actions (.github/workflows folder)

Container Layer

- Dockerfile used to build the container image

Deployment Layer

- Kubernetes cluster manages pods, services, and deployments

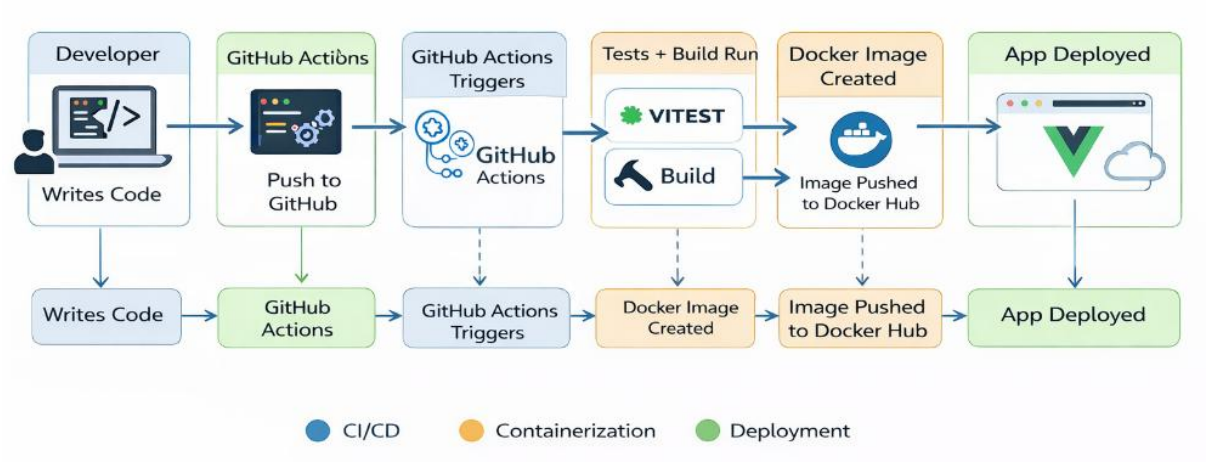


2.2 Pipeline Workflow

1. Developer writes code
2. Code is pushed to GitHub repository
3. CI pipeline is triggered automatically
4. Dependencies are installed and tests are executed
5. Application is built successfully
6. Docker image is created
7. Image is pushed to Docker Hub
8. Kubernetes pulls the image and deploys the application

2.3 Information Flow

Developer → GitHub Repo → GitHub Actions (CI) → Build + Test → Docker Image → Docker Hub → Kubernetes Cluster → Running App



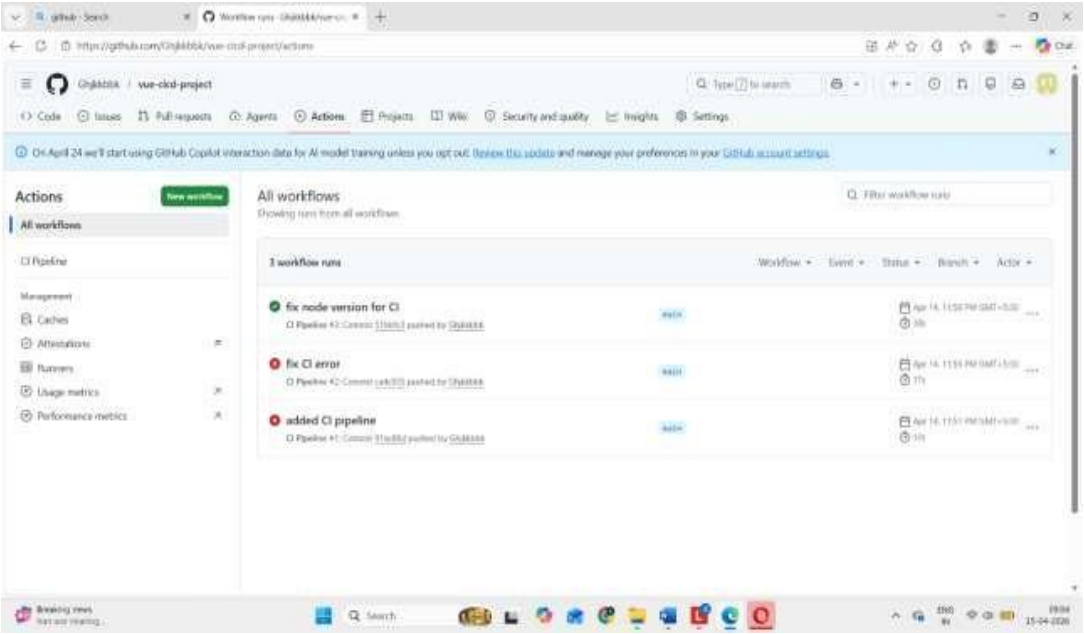
2.4 Component Design

1. GitHub Repository

Stores source code and version history; triggers CI workflow on commit

2. GitHub Actions

Executes CI/CD pipeline: build, test, publish, and deploy



3. Docker

Builds container image ensuring a uniform runtime environment

4. Docker Hub

Image registry storing and sharing built Docker images

5. Kubernetes

Orchestrates container deployment, scaling, and load balancing

2.5 Key Design Considerations

- **Automation:** Pipeline fully automated to minimize manual effort
- **Scalability:** Deployment scaling managed by Kubernetes
- **Reliability:** Deployments are repeatable and predictable
- **Consistency:** Docker ensures identical environments
- **Speed:** CI/CD reduces delivery and feedback time

2.6 API Catalogue

This is a **frontend deployment pipeline**, so the API layer is minimal.

API	Description
/	Loads Vue.js application
/health (optional)	Health check endpoint

3. Tools Used

- **GitHub** — Version Control System
- **GitHub Actions** — CI/CD Workflow Automation
- **Docker** — Containerization
- **Docker Hub** — Container Registry
- **Kubernetes** — Orchestration and Deployment

4. References

- GitHub Documentation
- Docker Documentation
- Kubernetes Documentation